

MINICHALLENGE 6

Sim → Physical Robot Migration Backlog

ROS 2 Humble · Puzzlebot · A1 RPLidar · Bug0 / Bug2 Navigation

Deadline: Tuesday 2 June 2026 — Video 4 PM CMT | Presentation 3 June 1 PM CMT

1. Executive Summary

This document is the complete migration backlog for taking minichallenge6_new_test from Gazebo simulation to the physical Puzzlebot with A1 RPLidar. It covers every required code change, the reasoning behind each, step-by-step procedures, expected results, and a 3-sprint plan sized to the Tuesday 2 June deadline.

Challenge scope: Part 1 — camera EKF localisation with ArUco markers (handled by a separate node to be developed). Part 2 — autonomous closed-loop navigation with Bug0 or Bug2 and obstacle avoidance (this repo). The optional part combines both.

Critical path: T-01 (QoS fix) → T-03/T-04 (real launch files) → T-07 (TF/laser_frame) → T-08 (LiDAR index) → T-13 (waypoints). Everything else is either parallel or polish.

2. Simulation vs Physical Robot: Key Differences

Item	Sim Behavior	Physical Robot Behavior
Encoder source	joint_states (Gazebo plugin)	/VelocityEncR & /VelocityEncL — Float32 via micro-ROS
QoS on encoder topics	Default RELIABLE (not critical in sim)	BEST_EFFORT + VOLATILE (must match micro-ROS publisher)
Odometry source	Gazebo diff_drive plugin → /odom	puzzlebot_localization.py integrates encoder input → /odom
TF odom → base_footprint	Gazebo + localisation.py (potential conflict)	Single source: joint_state_publisher node broadcasts TF
LiDAR frame_id	Simulated sensor frame (URDF-defined)	laser_frame (rplidar_ros default, set in launch arg)
Launch entry point	challenge.launch.py / challengeBug2.launch.py	challenge_real.launch.py / challengeBug2_real.launch.py (new)
use_sim_time	true	false
Gazebo nodes	gzserver, gzclient, spawn_entity	None — robot is live hardware
Joint names in joint_states	wheel_r_joint / wheel_l_joint	right_wheel_joint / left_wheel_joint (from real URDF xacro)

3. Expected Topic & TF Graph (Physical Robot)

All nodes running on the Jetson unless noted. The laptop is not required for Part 2 of the challenge.

```
micro-ROS (MCU Jetson)
pub → /VelocityEncR (Float32, BEST_EFFORT)
```

```

pub → /VelocityEncL (Float32, BEST_EFFORT)
sub ← /cmd_vel

localisation.py
sub ← /VelocityEncR, /VelocityEncL
pub → /odom (nav_msgs/Odometry, 50 Hz)

URDF_tfs.py
sub ← /odom
sub ← /wl, /wr (Float32 – verify topic names)
pub → /joint_states (JointState)
broadcast TF: odom → base_footprint

rplidar_node
pub → /scan (LaserScan, frame_id: laser_frame, ~7-10 Hz)

robot_state_publisher
sub ← /joint_states
broadcast static TF: base_footprint→base_link→lidar_base_link→laser_frame

bug0_node / bug2_node
sub ← /odom, /scan, /goal
pub → /cmd_vel

```

4. Migration Backlog

14 tasks. Priority legend: Blocker (must complete for robot to run at all), High (required for challenge compliance), Medium (stability/robustness), Low (cleanup).

#	File / Target	Change Required	Priority	Expected Result	Procedure	Risk
T-01	localisation.py	Replace JointState subscriber with Float32 subs on /VelocityEncR & /VelocityEncL. Add BEST_EFFORT QoS. Remove joint_callback.	Blocker	/odom published at 20 Hz with correct integrated pose. No silent zero-velocity condition.	1. Import Float32, QoSProfile, ReliabilityPolicy, DurabilityPolicy. 2. Create qos_sensor (BEST_EFFORT, VOLATILE, depth=10). 3. Replace joint_sub with two Float32 subs. 4. Add _cb_wr /	High

					_cb_wl callbacks. 5. Verify /odom with ros2 topic echo.	
T-02	localisation.py	Add covariance publishing to /odom (already computed as self.sigma — just map it into odom.pose.covariance[36]).	High	Covariance ellipsoids visible in RViz as required by challenge Part 1.	Copy sigma → covariance mapping pattern already in the file (indices 0,1,5,6,7,11,30,31,35).	Medium
T-03	launch/challenge_real.launch.py (NEW FILE)	Physical launch: robot_state_publisher (use_sim_time=false) + micro_ros_agent + rplidar_a1 + localisation + URDF_tfs + bug0_node. No Gazebo.	Blocker	All nodes alive on hardware. ros2 node list shows localisation, URDF_tfs, bug0_node. /scan and /odom echoing.	1. Copy challenge.launch.py. 2. Remove gazebo_launch include. 3. Add rplidar_a1_launch include (frame_id=laser_frame, port=/dev/rplidar). 4. Add micro_ros_agent include. 5. Set use_sim_time=false on all nodes.	High
T-04	launch/challengeBug2_real.launch.py (NEW FILE)	Same as T-03 but with bug2_node instead of bug0_node.	Blocker	Same as T-03 with bug2_node running.	Clone T-03 result, swap bug0_node → bug2_node executable.	High
T-05	URDF_tfs.py	Confirm /wl and /wr topic names match physical robot (micro-ROS publishes them as /wl and /wr per reference repo). If physical uses /VelocityEncL /VelocityEncR, update subscriber topics.	High	Wheel angle integration runs correctly. Joint states animate in RViz.	1. On physical robot: ros2 topic list after micro_ros_agent starts. 2. If topics differ, update wl_sub and wr_sub topic strings in URDF_tfs.py.	Medium

T - 0 6	URDF_tfs.py	Ensure URDF_tfs is the ONLY broadcaster of odom → base_footprint. Remove any static TF in localisation.py that broadcasts this frame.	High	Single TF source. No 'extrapolation into the past' errors in RViz.	1. Confirm localisation.py does not call tf_broadcaster.sendTransform for odom → base_footprint. 2. If it does, remove that call — URDF_tfs.py owns that edge.	Medium
T - 0 7	puzzlebot.urdf (or add xacro)	Verify laser_frame link exists in the URDF and is parented to base_link at the correct offset. If the URDF uses a different name, add a static_transform_publisher in the real launch.	High	ros2 run tf2_ros tf2_echo base_link laser_frame returns a valid transform. /scan visualizes correctly in RViz.	1. grep -i laser urdf/puzzlebot.urdf. 2. If laser_frame missing: add static_transform_publisher node in real launch: args='0.0 0.0 0.07 0 0 0 base_link laser_frame'.	Medium
T - 0 8	bug0_node.py / bug2_node.py	Verify LiDAR index 180 = front on physical A1. RPLidar A1 scans CCW from 0° — depending on mounting, index 0 may be front instead of 180.	High	Wall regions ('front','fright','right','fleft','left') map to correct physical directions. Robot turns away from obstacles correctly.	1. Mount robot facing a wall ~0.3m away. 2. ros2 topic echo /scan grep -A1 ranges. 3. Check which index has min value (should be ~160-200 if mounting is standard). 4. Adjust region slices if front is at index 0 instead of 180.	Medium
T - 0 9	bug0_node.py / bug2_node.py	Re-tune d_thresh (obstacle distance), v_max, w_max, k_rho, k_alpha on physical hardware. Sim values (d_thresh=0.4	Medium	Robot navigates physical track without wall collisions. Smooth arc during wall following.	1. Start with d_thresh=0.40, v_max=0.15, w_max=0.5. 2. Run single-target test. 3. Increase d_thresh if robot clips walls. 4. Adjust v_max based	Low

		5, v_max=0.2) may cause collisions or sluggish response on real floor.			on floor friction. 5. Log iterations.	
T - 1 0	launch files (all)	Add udev symlinks for /dev/rplidar and /dev/microros. Without them serial port changes on reconnect.	Medium	RPLidar and micro-ROS agent connect reliably on every boot.	Follow MIGRATION_GUIDE.md §3: add /etc/udev/rules.d/99-puzzlebot.rules with ATTRS{serial} for each device. Run udevadm reload.	Low
T - 1 1	robot_markers.py	Confirm this file is NOT launched on physical robot. It broadcasts a fake circular motion TF that will conflict with URDF_tfs.py.	Medium	No TF conflicts. RViz shows correct robot position.	1. Grep all launch files for robot_markers. 2. If present in any real launch, remove it.	Low
T - 1 2	odometry_node.py	Determine if odometry_node.py is still used. It publishes /pose (Pose2D) which is not consumed by any bug node. Likely dead code on real robot.	Low	No redundant node consuming CPU. Cleaner topic graph.	1. ros2 topic info /pose — check subscriber count. 2. If zero subscribers, remove from launch.	Minimal
T - 1 3	challenge goals	Add at least 4 waypoint targets for Part 2. Targets must not be collinear, not too close, and the path must circumnavigate obstacles. Currently no waypoint	Blocker	Bug0/Bug2 executes a closed trajectory visiting ≥4 waypoints with obstacle avoidance. Challenge Part 2 complete.	1. Measure physical track layout. 2. Select 4+ (x,y) target points on a closed loop. 3. Add a waypoint_sequencer node or extend bug nodes to accept a list of goals. 4.	High

		sequence is defined.			Publish each goal to /goal after previous goal is reached.	
T - 1 4	localisation.py	Add RViz covariance ellipsoid markers (visualization_msgs/Marker SPHERE with scale from sigma diagonal) as required by challenge spec.	High	Confidence ellipsoids visible in RViz shrinking when marker detected, growing when lost. Challenge Part 1 visual requirement met.	1. Create a Marker publisher in localisation.py. 2. On each update_localisation call, publish SPHERE marker with scale.x=sigma[0,0], scale.y=sigma[1,1], frame=odom.	Medium

5. Sprint Plan (3 Sprints to Deadline)

Sprint length is 1 day or less given the Tuesday deadline. No story points — just tasks grouped by logical dependency and testability.

Sprint	Tasks	Goals	Done When
Sprint 1 — Core Migration Today (31 May)	T-01, T-03, T-04, T-05, T-07	Physical robot boots with corrected localisation. /odom and /scan alive. TF tree valid.	ros2 topic hz /odom ≥ 20 Hz ros2 topic hz /scan ≥ 7 Hz tf2_echo odom base_footprint resolves tf2_echo base_link laser_frame resolves
Sprint 2 — Navigation Validation 1 June (AM)	T-06, T-08, T-09, T-11, T-13	Bug0 and Bug2 drive the robot to a single target without collision. LiDAR regions verified against physical mounting.	Single-point test: robot reaches (x,y) within goal_tolerance No collision with obstacle Wall-following triggers and exits correctly
Sprint 3 — Full Challenge 1 June (PM) → 2 June	T-02, T-13, T-14, T-10, T-12	4-waypoint closed loop complete. Covariance ellipsoids in RViz. All challenge requirements met. Video recorded.	Robot completes full closed trajectory ≥ 1 clean run Ellipsoids visible and evolving in RViz Video recorded and uploaded to YouTube by 4 PM June 2

6. Detailed Code Changes & Procedures

T-01 · localisation.py — Full Patch

Replace the JointState subscriber block with this:

```
from std_msgs.msg import Float32
from rclpy.qos import QoSProfile, ReliabilityPolicy, DurabilityPolicy

# In __init__, REPLACE joint_sub line with:
qos_sensor = QoSProfile(
    reliability=ReliabilityPolicy.BEST_EFFORT,
    durability=DurabilityPolicy.VOLATILE,
    depth=10,
)
self.create_subscription(Float32, '/VelocityEncR', self._cb_wr, qos_sensor)
self.create_subscription(Float32, '/VelocityEncL', self._cb_wl, qos_sensor)

# ADD these callbacks (remove joint_callback entirely):
def _cb_wr(self, msg): self.wr = msg.data
def _cb_wl(self, msg): self.wl = msg.data
```

Why BEST_EFFORT: micro-ROS on the Jetson MCU publishes encoder topics with BEST_EFFORT QoS. ROS 2 default subscription QoS is RELIABLE, which is incompatible — the subscriber receives zero messages without any error. This is the most common silent failure in Puzzlebot sim → real migration.

T-07 · URDF — laser_frame verification

Run this to check if laser_frame exists in the current URDF:

```
grep -i laser urdf/puzzlebot.urdf
```

If laser_frame is absent, add a static_transform_publisher in the real launch file:

```
<node pkg="tf2_ros" exec="static_transform_publisher"
  name="laser_tf"
  args="0.0 0.0 0.07 0 0 0 base_link laser_frame" />
```

The offset (0, 0, 0.07) matches the lidar_base_pos_z from the reference xacro. Adjust if the physical mounting differs.

T-08 · LiDAR Index Convention Verification

The current bug nodes assume index 180 = front of robot (i.e., the scan starts behind the robot at index 0). The RPLidar A1 default orientation depends on physical mounting.

Verification procedure:

- Place robot 0.3m from a wall, facing it directly.
- `ros2 topic echo /scan --once | python3 -c "import sys,json; d=json.load(sys.stdin); print(d['ranges'][160:200])"`
- If that slice shows low values (~0.3m) → mounting is standard, no change needed.
- If index 0 shows low values → front is at 0, not 180. Update region slices: `front=[-20:20] mod 360 = [340:360]+[0:20]`.

T-13 · Waypoint Sequencer Pattern

The challenge requires ≥ 4 targets on a closed loop with no collinear points. Simplest approach: extend either bug node to accept a list and auto-advance on goal reach.

```
# Add to bug0_node / bug2_node __init__:
self.waypoints = [(1.0,0.0),(1.0,1.5),(-0.5,1.5),(-0.5,0.0)] # meters
self.wp_idx = 0

# In control_loop, when STOP state is reached:
self.wp_idx = (self.wp_idx + 1) % len(self.waypoints)
self.target_x, self.target_y = self.waypoints[self.wp_idx]
self.goal_received = True
self.change_state('GO_TO_GOAL')
```

Note: Coordinates are in the odom frame. Measure the physical track first and set waypoints accordingly. Use a closed loop (last waypoint connects back to start) to satisfy the 'closed trajectory' requirement.

7. Troubleshooting Reference

Symptom	Likely Cause	Fix
/odom not publishing or always zero	QoS mismatch — encoder subscriber using RELIABLE vs micro-ROS BEST Effort publisher	T-01: Add BEST_EFFORT QoSProfile to VelocityEncR/L subscriptions
TF extrapolation errors in RViz	Multiple nodes broadcasting odom → base_footprint (Gazebo + URDF_tfs running simultaneously)	T-06: Ensure only URDF_tfs.py broadcasts this edge. Remove from real launch any Gazebo-side TF.
Robot drives into walls instead of avoiding them	LiDAR index convention mismatch — 'front' region mapped to wrong angles	T-08: Verify index 160:200 = front. Adjust region slices if needed.
/scan not visible in RViz / TF error for laser_frame	laser_frame link missing from URDF, or rplidar publishing with different frame_id	T-07: Check URDF for laser_frame. Add static_transform_publisher or set frame_id arg in rplidar launch.
rplidar_node fails to start	/dev/rplidar symlink does not exist	T-10: Create udev rule. udevadm

		control --reload-rules && udevadm trigger, then reconnect device.
Robot oscillates / spins in place	k_alpha too high, or d_thresh too aggressive for real floor friction	T-09: Reduce k_alpha to 1.0, increase goal_tolerance to 0.20, lower w_max to 0.4 first.
Covariance ellipsoids not showing in RViz	T-02 / T-14 not implemented: covariance not published or marker not broadcasted	T-02: Map self.sigma into odom.pose.covariance. T-14: Add Marker publisher with scale from sigma diagonal.

8. Pre-Demo Verification Checklist

Run through this before recording the video. Every item must be green.

- `ros2 topic hz /scan` → ≥ 7 Hz
- `ros2 topic hz /odom` → ≥ 20 Hz
- `ros2 topic hz /VelocityEncR` → data flowing
- `ros2 run tf2_tools view_frames` → no disconnected frames
- `ros2 run tf2_ros tf2_echo odom base_footprint` → valid
- `ros2 run tf2_ros tf2_echo base_link laser_frame` → valid
- Bug0 single-target test → robot reaches goal without collision
- Bug2 single-target test with obstacle in path → wall following activates and exits
- 4-waypoint closed loop → robot completes full trajectory
- RViz → covariance ellipsoids visible and evolving
- Video recorded (3-4 min, English, includes workflow diagram)
- YouTube link ready for submission by 4 PM CMT June 2